

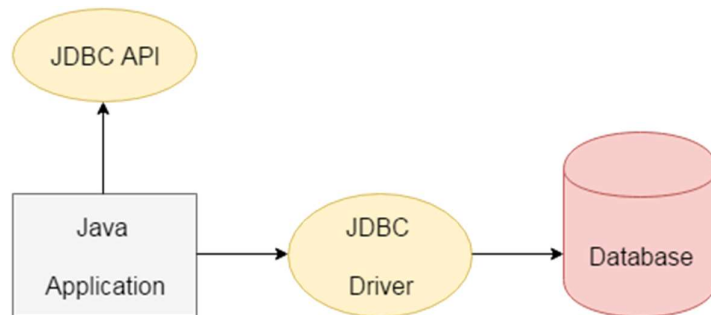
Java JDBC Tutorial

JDBC stands for Java Database Connectivity. JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers to connect with the database. There are four types of JDBC drivers:

- JDBC-ODBC Bridge Driver,
- Native Driver,
- Network Protocol Driver, and
- Thin Driver

We have discussed the above four drivers in the next chapter.

We can use JDBC API to access tabular data stored in any relational database. By the help of JDBC API, we can save, update, delete and fetch data from the database. It is like Open Database Connectivity (ODBC) provided by Microsoft.



The current version of JDBC is 4.3. It is the stable release since 21st September, 2017. It is based on the X/Open SQL Call Level Interface. The **java.sql** package contains classes and interfaces for JDBC API. A list of popular *interfaces* of JDBC API are given below:

- Driver interface
- Connection interface
- Statement interface
- PreparedStatement interface
- CallableStatement interface
- ResultSet interface
- ResultSetMetaData interface
- DatabaseMetaData interface

- RowSet interface

A list of popular *classes* of JDBC API are given below:

- DriverManager class
- Blob class
- Clob class
- Types class

Why Should We Use JDBC

Before JDBC, ODBC API was the database API to connect and execute the query with the database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

We can use JDBC API to handle database using Java program and can perform the following activities:

1. Connect to the database
2. Execute queries and update statements to the database
3. Retrieve the result received from the database.

JDBC Driver

1. JDBC Drivers

1. JDBC-ODBC bridge driver

2. Native-API driver

3. Network Protocol driver

4. Thin driver

JDBC Driver is a software component that enables java application to interact with the database. There are 4 types

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)
4. Thin driver (fully java driver)

1) JDBC-ODBC bridge driver

- 2) Microsoft's [ODBC](#) is the most commonly used driver to connect to the [database](#) as it can connect to almost all databases on most of the platforms. However, [ODBC](#) uses the concept of pointers and other constructs that are not supported by [Java](#).
- 3)
- 4) Therefore, [JDBC](#)-ODBC bridge driver was developed which translates the [JDBC](#) API to the ODBC API and vice versa. This bridge acts as interface that enables all [DBMS](#) which support ODBC (Open Data Base Connectivity) to interact with [Java](#) Applications. JDBC-ODBC bridge is implemented as a class file and a native library. The name of the class file is JdbcOdbc.class. On windows platform, native library called **JDBCODBC.DLL** is used.

The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts java calls into the ODBC function calls. This is now discouraged because of thin driver.

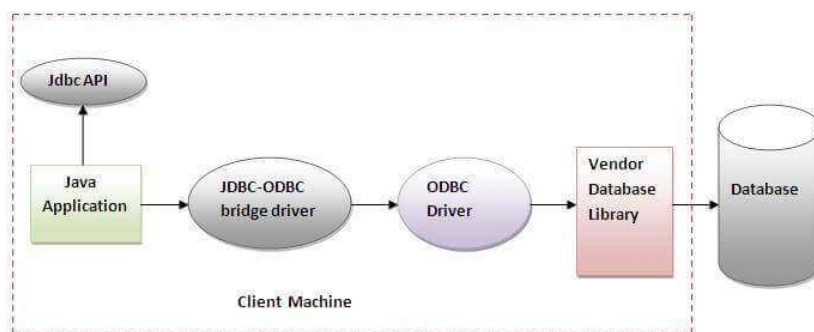


Figure- JDBC-ODBC Bridge Driver

In Java 8, the JDBC-ODBC Bridge has been removed.

Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

Advantages:

- easy to use.
- can be easily connected to any database.

Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

Servlets | Servlet Tutorial

Servlet technology is used to create a web application (resides at server side and generates a dynamic web page).

Servlet technology is robust and scalable because of java language. Before Servlet, CGI (Common Gateway Interface) scripting language was common as a server-side programming language. However, there were many disadvantages to this technology. We have discussed these disadvantages below.

There are many interfaces and classes in the Servlet API such as Servlet, GenericServlet, HttpServlet, ServletRequest, ServletResponse, etc.

What is a Servlet?

Servlet can be described in many ways, depending on the context.

- Servlet is a technology which is used to create a web application.
- Servlet is an API that provides many interfaces and classes including documentation.
- Servlet is an interface that must be implemented for creating any Servlet.
- Servlet is a class that extends the capabilities of the servers and responds to the incoming requests. It can respond to any requests.
- Servlet is a web component that is deployed on the server to create a dynamic web page.

What is a web application?

A web application is an application accessible from the web. A web application is composed of web components like Servlet, JSP, Filter, etc. and other elements such as HTML, CSS, and JavaScript. The web components typically execute in Web Server and respond to the HTTP request.

CGI (Common Gateway Interface)

CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.

Disadvantages of CGI

There are many problems in CGI technology:

1. If the number of clients increases, it takes more time for sending the response.
2. For each request, it starts a process, and the web server is limited to start processes.
3. It uses platform dependent language e.g. [C](#)

, [C++](#)

, [perl](#)

.

Advantages of Servlet

There are many advantages of Servlet over CGI. The web container creates threads for handling the multiple requests to the Servlet. Threads have many benefits over the Processes such as they share a common memory area, lightweight, cost of communication between the threads are low. The advantages of Servlet are as follows:

1. **Better performance:** because it creates a thread for each request, not process.
2. **Portability:** because it uses Java language.
3. **Robust:** [JVM](#)

manages Servlets, so we don't need to worry about the memory leak, [garbage collection](#)

, etc.

4. **Secure:** because it uses java language.

Life Cycle of a Servlet (Servlet Life Cycle)

1. Life Cycle of a Servlet

1. Servlet class is loaded

2. Servlet instance is created

3. init method is invoked

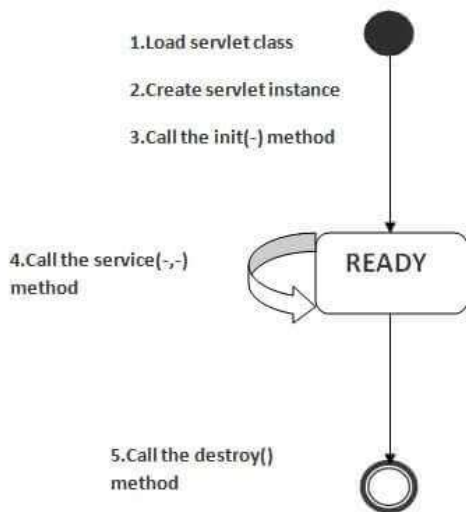
4. service method is invoked

5. destroy method is invoked

Life Cycle of a Servlet (Servlet Life Cycle)

The web container maintains the life cycle of a servlet instance. Let's see the life cycle of the servlet:

1. Servlet class is loaded.
2. Servlet instance is created.
3. init method is invoked.
4. service method is invoked.
5. destroy method is invoked.



As displayed in the above diagram, there are three states of a servlet: new, ready and end. The servlet is in new state if servlet instance is created. After invoking the `init()` method, Servlet comes in the ready state. In the ready state, servlet performs all the tasks. When the web container invokes the `destroy()` method, it shifts to the end state.

1) Servlet class is loaded

The classloader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

2) Servlet instance is created

The web container creates the instance of a servlet after loading the servlet class. The servlet instance is created only once in the servlet life cycle.

3) init method is invoked

The web container calls the `init` method only once after creating the servlet instance. The `init` method is used to initialize the servlet. It is the life cycle method of the `javax.servlet.Servlet` interface. Syntax of the `init` method is given below.

1. **public void** `init(ServletConfig config)` **throws** `ServletException`
-

4) service method is invoked

The web container calls the service method each time when request for the servlet is received. If servlet is not initialized, it follows the first three steps as described above then calls the service method. If servlet is initialized, it calls the service method. Notice that servlet is initialized only once. The syntax of the service method of the Servlet interface is given below:

1. **public void** service(ServletRequest request, ServletResponse response)
 2. **throws** ServletException, IOException
-

5) destroy method is invoked

The web container calls the destroy method before removing the servlet instance from the service. It gives the servlet an opportunity to clean up any resource for example memory, thread etc. The syntax of the destroy method of the Servlet interface is given below:

JSP Tutorial

JSP technology is used to create web application just like Servlet technology. It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL, etc.

A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than Servlet because we can separate designing and development. It provides some additional features such as Expression Language, Custom Tags, etc.

Advantages of JSP over Servlet

There are many advantages of JSP over the Servlet. They are as follows:

1) Extension to Servlet

JSP technology is the extension to Servlet technology. We can use all the features of the Servlet in JSP. In addition to, we can use implicit objects, predefined tags, expression language and Custom tags in JSP, that makes JSP development easy.

2) Easy to maintain

JSP can be easily managed because we can easily separate our business logic with presentation logic. In Servlet technology, we mix our business logic with the presentation logic.

3) Fast Development: No need to recompile and redeploy

If JSP page is modified, we don't need to recompile and redeploy the project. The Servlet code needs to be updated and recompiled if we have to change the look and feel of the application.

4) Less code than Servlet

In JSP, we can use many tags such as action tags, JSTL, custom tags, etc. that reduces the code. Moreover, we can use EL, implicit objects, etc.

The Lifecycle of a JSP Page

The JSP pages follow these phases:

- Translation of JSP Page
- Compilation of JSP Page
- Classloading (the classloader loads class file)
- Instantiation (Object of the Generated Servlet is created).
- Initialization (the container invokes `jspInit()` method).
- Request processing (the container invokes `_jspService()` method).
- Destroy (the container invokes `jspDestroy()` method).

Note: `jspInit()`, `_jspService()` and `jspDestroy()` are the life cycle methods of JSP.

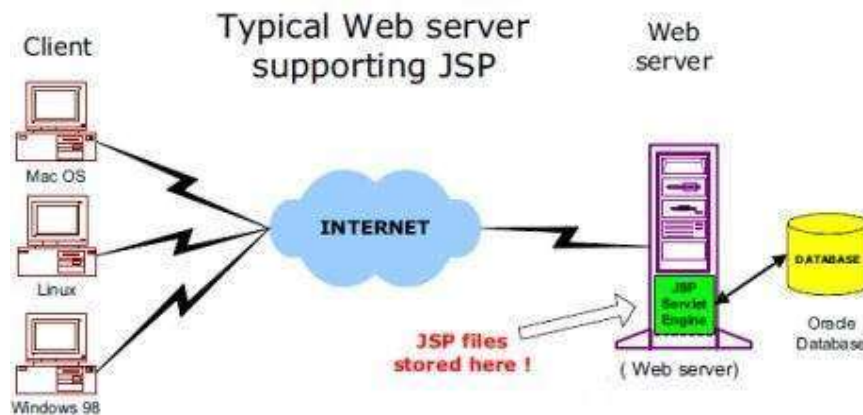
As depicted in the above diagram, JSP page is translated into Servlet by the help of JSP translator. The JSP translator is a part of the web server which is responsible for translating the JSP page into Servlet. After that, Servlet page is compiled by the compiler and gets converted into the class file. Moreover, all the processes that happen in Servlet are performed on JSP later like initialization, committing response to the browser and destroy.

JSP - Architecture

The web server needs a JSP engine, i.e, a container to process JSP pages. The JSP container is responsible for intercepting requests for JSP pages. This tutorial makes use of Apache which has built-in JSP container to support JSP pages development.

A JSP container works with the Web server to provide the runtime environment and other services a JSP needs. It knows how to understand the special elements that are part of JSPs.

Following diagram shows the position of JSP container and JSP files in a Web application.



JSP Processing

The following steps explain how the web server creates the Webpage using JSP –

- As with a normal page, your browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with **.jsp** instead of **.html**.
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to `println( )` statements and all JSP elements are converted to Java code. This code implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- A part of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. The output is further passed on to the web server by the servlet engine inside an HTTP response.
- The web server forwards the HTTP response to your browser in terms of static HTML content.

- Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page.

All the above mentioned steps can be seen in the following diagram –

Typically, the JSP engine checks to see whether a servlet for a JSP file already exists and whether the modification date on the JSP is older than the servlet. If the JSP is older than its generated servlet, the JSP container assumes that the JSP hasn't changed and that the generated servlet still matches the JSP's contents. This makes the process more efficient than with the other scripting languages (such as PHP) and therefore faster.

So in a way, a JSP page is really just another way to write a servlet without having to be a Java programming wiz. Except for the translation phase, a JSP page is handled exactly like a regular servlet.

Life cycle of JSP

A Java Server Page life cycle is defined as the process that started with its creation which later translated to a servlet and afterward servlet lifecycle comes into play. This is how the process goes on until its destruction.

Following steps are involved in the JSP life cycle:

- Translation of JSP page to Servlet
- Compilation of JSP page(Compilation of JSP into test.java)
- Classloading (test.java to test.class)
- Instantiation(Object of the generated Servlet is created)
- Initialization(jspInit() method is invoked by the container)
- Request processing(_jspService()is invoked by the container)
- JSP Cleanup (jspDestroy() method is invoked by the container)

We can override jspInit(), jspDestroy() but we can't override _jspService() method.

Translation of JSP page to Servlet :

This is the first step of the JSP life cycle. This translation phase deals with the Syntactic correctness of JSP. Here test.jsp file is translated to test.java.

Compilation of JSP page :

Here the generated java servlet file (test.java) is compiled to a class file (test.class).

Classloading :

Servlet class which has been loaded from the JSP source is now loaded into the container.

Instantiation :

Here an instance of the class is generated. The container manages one or more instances by providing responses to requests.

Initialization :

`jspInit()` method is called only once during the life cycle immediately after the generation of Servlet instance from JSP.

Request processing :

`_jspService()` method is used to serve the raised requests by JSP. It takes request and response objects as parameters. This method cannot be overridden.

JSP Cleanup :

In order to remove the JSP from the use by the container or to destroy the method for servlets `jspDestroy()` method is used. This method is called once, if you need to perform any cleanup task like closing open files, releasing database connections `jspDestroy()` can be overridden.

MVC Architecture in Java

The Model-View-Controller (MVC) is a well-known [design pattern](#)

in the web development field. It is way to organize our code. It specifies that a program or application shall consist of data model, presentation information and control information. The MVC pattern needs all these components to be separated as different objects.

In this section, we will discuss the MVC Architecture in Java, alongwith its advantages and disadvantages and examples to understand the implementation of MVC in Java.

What is MVC architecture in Java?

The model designs based on the MVC architecture follow MVC design pattern. The application logic is separated from the user interface while designing the software using model designs.

The MVC pattern architecture consists of three layers:

Model: It represents the business layer of application. It is an object to carry the data that can also contain the logic to update controller if data is changed.

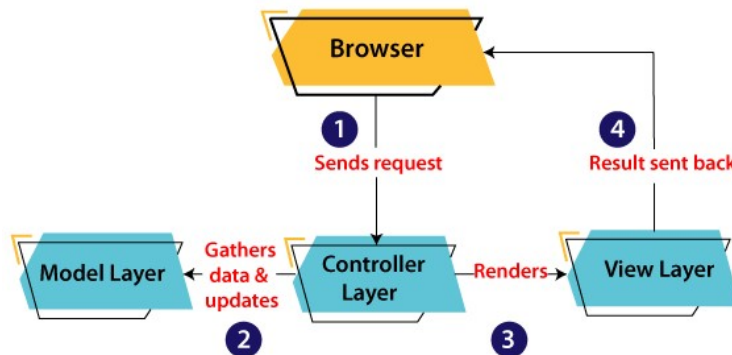
- **View:** It represents the presentation layer of application. It is used to visualize the data that the model contains.

- **Controller:** It works on both the model and view. It is used to manage the flow of application, i.e. data flow in the model object and to update the view whenever data is changed.

In Java Programming, the Model contains the simple **Java classes**

, the View used to display the data and the Controller contains the **servlets**

. Due to this separation the user requests are processed as follows:



1. A client (browser) sends a request to the controller on the server side, for a page.
2. The controller then calls the model. It gathers the requested data.
3. Then the controller transfers the data retrieved to the view layer.
4. Now the result is sent back to the browser (client) by the view.

Advantages of MVC Architecture

The advantages of MVC architecture are as follows:

- MVC has the feature of scalability that in turn helps the growth of application.
- The components are easy to maintain because there is less dependency.
- A model can be reused by multiple views that provides reusability of code.
- The developers can work with the three layers (Model, View, and Controller) simultaneously.
- Using MVC, the application becomes more understandable.
- Using MVC, each layer is maintained separately therefore we do not require to deal with massive code.
- The extending and testing of application is easier.

Introduction and Working of Struts Web Framework

Struts is used to create a web applications based on servlet and JSP. Struts depend on the MVC (Model View Controller) framework. Struts application is a genuine web application. Struts are thoroughly useful in building J2EE (Java 2 Platform, Enterprise Edition) applications because struts takes advantage of J2EE design patterns. Struts follows these J2EE design patterns including MVC. In struts, the composite view manages the layout of its sub-views and can implement a template, making persistent look and feel easier to achieve and customize across the entire application. A composite view is made up by using other reusable sub views such that a small change happens in a sub-view is automatically updated in every composite view.

Struts consists of a set of own custom tag libraries. Struts are based on MVC framework which is pattern oriented and includes JSP custom tag libraries. Struts also supports utility classes.

Features of Struts: Struts has the following features:

- Struts encourages good design practices and modeling because the framework is designed with “time-proven” design patterns.
- Struts is almost simple, so easy to learn and use.
- It supports many convenient features such as input validation and internationalization.
- It takes much of the complexity out as instead of building your own MVC framework, you can use struts.
- Struts is very well integrated with J2EE.
- Struts has large user community.
- It is flexible and extensible, it is easy for the existing web applications to adapt the struts framework.
- Struts provide good tag libraries.
- It allows capturing input form data into javabean objects called Action forms.
- It also hand over standard error handling both programmatically and declaratively.

In the initialization phase, the controller rectify a configuration file and used it to deploy other control layer objects. Struts configuration is form by these objects combined together. The struts configuration defines among other things the action mappings for an application.

Struts controller servlet considers the action mappings and routes the HTTP requests to other components in the framework. Request is first delivered to an action and then to JSP. The mapping helps the controller to change HTTP requests into application actions. The action objects can handle the request from

and responds to the client (generally a web browser). Action objects have access to the applications controller servlet and also access to the servlet's methods. When delivering the control, an action objects can indirectly forward one or more share objects, including javabeans by establish them in the typical situation shared by java servlets.

Introduction to Struts Architecture

Struts architecture is made up by using the most popular concept in java applications which is MVC. MVC pattern is followed by struts which stands for Model View Controller that is essentially used when you want to bring the separation in your application between the application data, business logic and the way in which all this is presented to the user. The main advantage of having MVC architecture for struts is that the application becomes reusable and expressive. In case, if any sort of change is to be made when the problem occurs, we don't need to write the code solution from scratch. We can simply learn the pattern and modify it as per necessity.

Here we will see the MVC pattern followed by struts architecture and also see about various other components that form the architecture of struts.

Model View Controller

Before we see the components and architecture diagram of struts let us first see the conceptual base on which the architecture of struts is created that is the MVC pattern.

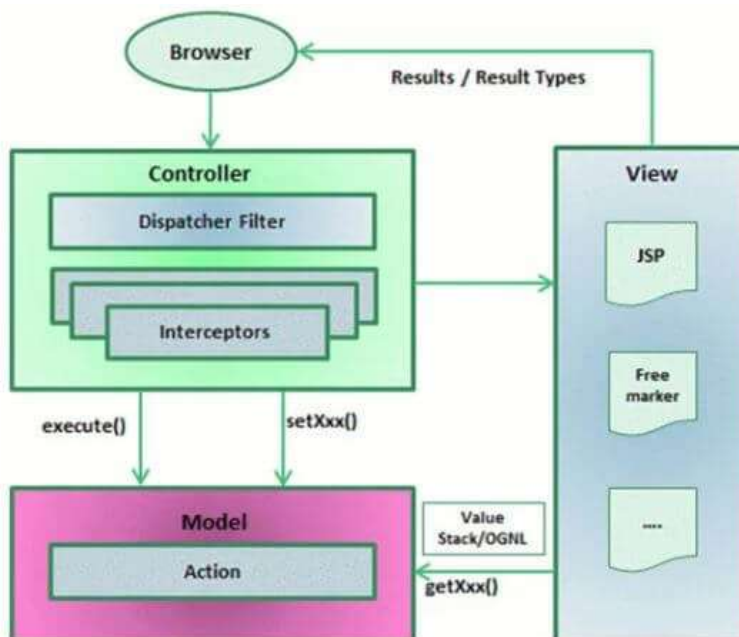
- **Model:** The model includes the business logic of the code where the operations related to data such as access to the data, validating the same and the way in which the data is to be saved in data stores or database is included as the responsibility of Model part. Before the data is sent to the database for saving, it is verified by the views present and if the structure of the data is proper then only it is further sent to the database for inserting the record or doing any other sort of manipulations.
- **View:** This part is responsible for all the things that the user will be able to view and also it will take all the required inputs from the user, generating the request that should be passed to the controller for handling and also accept the responses that are in turn sent by the controller which can be further

displayed on screen for the users. When using sprites, we can make the use of custom tag libraries, HTML, resources file and HTML when using the View component of the MVC structure.

- **Controller:** This component acts as the median between the other two components that is View and Model. It is responsible for managing and handling all the requests that are received by it from the view side that is user. As soon as the request arrives, a certain business logic code is executed by the controller which handles the model structure and after executing returns the proper data in the form of a model to the view. The parts included in the controller are struts-config.xml, ActionForm and ActionServlet.

Architecture of Struts

The diagram that will help us to demonstrate the architecture of struts is given below. Let us now see different components of struts architecture and how the flow works.



We can clearly observe the three broad components of the strut's architecture are Model, View and controller that form the high-level struts architecture. When they are further digging to get the details, we get a clearer view. Interceptors and dispatch servlet filters together try to implement the controller in struts. The model component is carried out by using the actions. The third component that is view is

generated by combining results and the type of results. The integration between view and controller and view and model is enabled by using OGNL and value stack which act as a linking and common thread.

Other than the components mentioned in the above diagram, struts architecture includes a lot of configuration related details that include the configuration for results, interceptors, actions, the web application and many other components details.

The life cycle of the request done by the user can be understood by studying the workflow of struts which can be derived from the above architecture diagram.

- The required resource is taken by the view and the user(client) further makes the request to the server in case of client-server architecture which contains the specification of the required resource.
- Proper action is taken after the filter dispatcher studies the request and decides the action to be taken.
- The functionalities related to interceptors that are configured such as file upload of validation are carried out.
- The action that is decided and selected after this is executed depending on the type of operation requested in the client request.
- If any sort of post-processing is required then the application of configured interceptors is carried out again.
- The last step includes the creation of the result by the view and returning the same as the response to the user (client).